

Customer Behavioral Segmentation System

Phase 3 Report — Multi-Feature Training, DBSCAN & Deployment

B.Tech Artificial Intelligence & Machine Learning

Field	Details
Project Title	Customer Behavioral Segmentation System
Phase	Phase 3 — Multi-Feature Training, DBSCAN & Console Deployment
Algorithms	K-Means Clustering + DBSCAN (both trained & deployed)
Feature Combinations	5 pairs across Annual_Spending, Purchase_Frequency, Returns_Count, Discount_Usage_Pct, Age
Deployment File	main.py — Menu-driven Python console application
Model Artifacts	10 KMeans .pkl + 10 DBSCAN .pkl + 5 scaler sets = 30 files
Platform	Python 3.x Jupyter Notebook VS Code pickle

Table of Contents

1. Problem Description
 - 1.1 Dataset Description
 - 1.2 Flow Diagram
2. Modules for Project Implementation
 - 2.1 Modules Used
 - 2.2 Platform Used
3. Machine Learning Models Used
4. Evaluation Metrics
5. Results and Visualizations
6. Conclusion and Future Scope
7. References

1. Problem Description

Phase 3 represents the final and most technically complete stage of the Customer Behavioral Segmentation System. Having established clean customer personas in Phase 2 using `Annual_Spending` and `Purchase_Frequency` alone, Phase 3 significantly expands the system in two directions: breadth and deployability. On the breadth side, it trains both K-Means and DBSCAN models across five different feature pair combinations, making the system flexible enough to segment customers from multiple behavioral angles — not just spending and frequency, but also return behavior, discount sensitivity, and age-spending relationships. On the deployment side, the trained models are serialized as pickle files and loaded by a fully functional menu-driven console application (`main.py`) that accepts live user inputs and predicts customer segments in real time.

The problem this phase solves is the gap between a trained notebook model and a usable system. A Jupyter notebook with a clustering result is useful for analysis, but it cannot be handed to a marketing analyst and asked 'can you check which segment this new customer belongs to?' Phase 3 bridges that gap. The `main.py` system lets any user — without knowing Python or machine learning — select an algorithm, select a feature combination, enter two customer values, and immediately receive a segment label such as 'High Value Customer' or 'Noise / Outlier.' This is what makes the project a system, not just an experiment.

DBSCAN adds a dimension that K-Means fundamentally cannot provide: outlier detection. In any real customer database, there will be anomalous customers who do not fit neatly into known behavioral groups — perhaps someone who makes 50 purchases a year but spends almost nothing, or someone whose return count is far outside the normal range. K-Means will force every such customer into the nearest cluster regardless, potentially polluting the cluster's profile. DBSCAN marks these as noise (label = -1) and handles them separately. This gives marketing teams a cleaner picture of their core segments and a list of anomalous customers who might warrant individual investigation.

1.1 Dataset Description

The dataset used throughout this project is `retail_finding_dataaa.csv`, a synthetic but realistically structured retail dataset containing 1000 customer records across 9 columns. Phase 1 EDA confirmed the dataset is clean — zero null values, zero duplicates — making it directly usable for machine learning without any data cleaning steps.

Column	Type	Description	Role in Phase 3
Customer_ID	Object (str)	Unique customer identifier (CUST0001...CUST1000)	Excluded from modeling — identifier only
Age	int64	Customer age in years (range: 18–70)	Used in feature pairs 4 and 5
Gender	object	Categorical: Male, Female, Non-Binary	Encoded in Phase 2; not used in Phase 3 feature pairs
Region	object	5 regions: North, South, East, West, Central	Not used in Phase 3 clustering
Annual_Spending	float64	Total annual spend in INR (range: ₹50–₹19,907)	Primary feature — used in pairs 1, 2, 4
Purchase_Frequency	int64	Number of purchases per year (range: 1–30)	Used in feature pairs 1, 3, 5
Discount_Usage_Pct	float64	% of purchases made with discount (5%–70%)	Used in feature pair 3
Returns_Count	int64	Number of items returned per year (0–10)	Used in feature pair 2
Preferred_Category	object	10 categories: Electronics, Books, Health, etc.	Not used in Phase 3 clustering

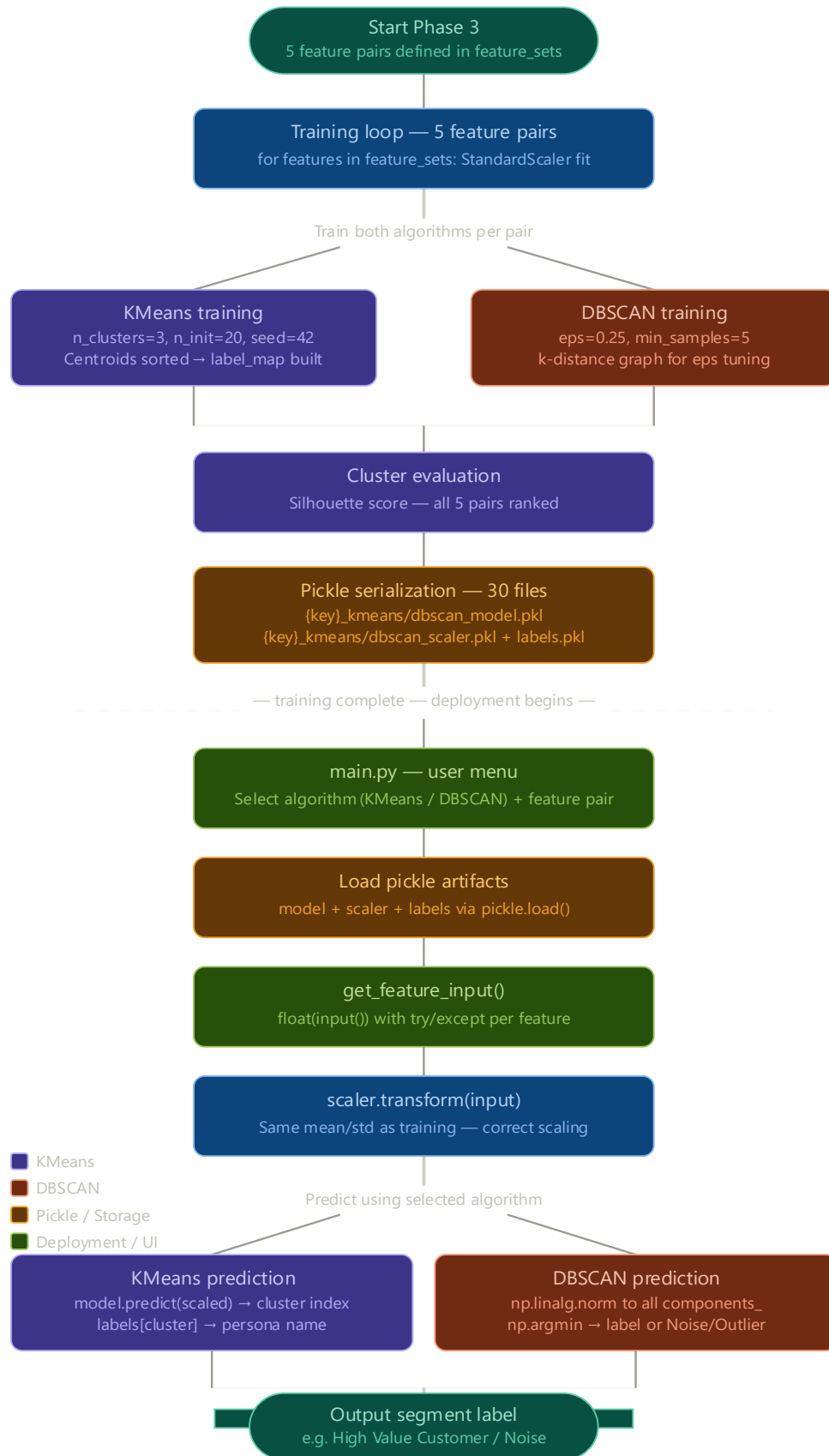
The five feature pairs used in Phase 3 training are derived from the `FEATURE_POOL = ['Age', 'Annual_Spending', 'Purchase_Frequency', 'Returns_Count', 'Discount_Usage_Pct']` and the `feature_sets` list. Each pair captures a different behavioral dimension:

Pair #	Feature 1	Feature 2	Behavioral Dimension Captured
1	Annual_Spending	Purchase_Frequency	Core purchasing power — primary segmentation axis
2	Annual_Spending	Returns_Count	High spenders who return frequently vs. those who don't
3	Purchase_Frequency	Discount_Usage_Pct	Frequency vs price-sensitivity — budget vs. loyal buyers
4	Age	Annual_Spending	Spending behavior across life stages
5	Age	Purchase_Frequency	Purchase habit patterns across age groups

1.2 Flow Diagram

The following flow diagram represents the complete Phase 3 pipeline, from data loading through model training to real-time deployment via the console system:

PHASE 3 — COMPLETE SYSTEM FLOW



2. Modules for Project Implementation

2.1 Modules Used

All modules used in Phase 3 are standard Python libraries, each serving a specific and essential function in the pipeline. Below is a complete breakdown of every module used across both the training notebook and the deployment system:

pandas

Pandas is the foundation of data handling in this project. It is used to load the CSV file using `pd.read_csv('retail_finding_dataa.csv')`, create the main DataFrame `df`, run all EDA operations (`df.info()`, `df.describe()`, `df.isnull().sum()`, `df.duplicated().sum()`), execute groupby aggregations for cluster profiling (`df.groupby('Persona').agg(...)`), generate skewness values (`df[nc].skew()`), and add computed columns like `df['Cluster']`, `df['Persona']`, `df['Age_Group']`, and `df['Age_Gender_Cluster']`. Without pandas, structured manipulation of 1000 customer records would require raw Python loops and dictionaries — pandas makes it efficient, readable, and production-grade.

NumPy

NumPy handles all low-level numerical computation. In the training notebook it backs the feature matrix creation (`X = df[features].dropna().values`), centroid inverse-transformation, and sorting logic (`np.argsort(centers[:, 0])`). In the deployment system `main.py`, NumPy is used for three critical operations: creating the input array (`np.array([values])`), computing Euclidean distances between the new customer point and all DBSCAN core points (`np.linalg.norm(data_scaled - core_point)`), and finding the closest core point (`np.argmin(distances)`). The DBSCAN prediction logic in `main.py` is entirely NumPy-driven.

scikit-learn — sklearn.cluster.KMeans

KMeans from `sklearn.cluster` is the primary clustering algorithm. In Phase 3 training, it is instantiated as `KMeans(n_clusters=3, n_init=20, random_state=42)` for each of the five feature pairs. The `n_clusters=3` is used in Phase 3 (compared to `k=4` in Phase 2) because Phase 3 is training a general-purpose prediction system where three tiers — Low Value Customer, Regular Customer, High Value Customer — are the standard business classification framework. `n_init=20` ensures the algorithm runs 20 independent restarts and picks the best result, significantly reducing the chance of convergence to a suboptimal local minimum.

scikit-learn — sklearn.cluster.DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) is instantiated as `DBSCAN(eps=0.25, min_samples=5)`. The `eps=0.25` parameter defines the neighborhood radius in the scaled feature space — any two points within 0.25 scaled units of each other are considered neighbors. `min_samples=5` means a point must have at least 5 neighbors within `eps` to qualify as a core point. The k-distance graph (plotted using `NearestNeighbors` with `n_neighbors=5`) is used to empirically validate this `eps` choice. After fitting, `dbscan.labels_` contains -1 for noise points and integer cluster IDs for all other points. The `model.components_` attribute stores all core points — this is what `main.py` iterates over during prediction.

scikit-learn — sklearn.preprocessing.StandardScaler

`StandardScaler` is applied to every feature pair before both K-Means and DBSCAN training. It is fit only on the training data using `scaler.fit_transform(X)` and saved as a pickle file. At prediction time in `main.py`, the same saved scaler is loaded and used with `scaler.transform(data)` on the new user input. This ensures the new data point is scaled using the exact same mean and standard deviation computed on the original 1000-customer training set — which is the correct way to apply scaling in a deployed system. Using a fresh scaler at inference time would produce incorrect results because the mean and std would differ.

scikit-learn — sklearn.metrics.silhouette_score

`silhouette_score(X_scaled, kmeans.labels_)` is called after fitting `KMeans` on each feature pair to compute the Silhouette Score. This provides per-combination validation — the team can see which feature pair produces the best-separated clusters. It is also used in Phase 2 for the primary cluster validation. The score is printed to console and recorded for analysis.

sklearn.neighbors.NearestNeighbors

Used exclusively to generate the k-distance graph for DBSCAN `eps` parameter tuning. `NearestNeighbors(n_neighbors=5).fit(X_scaled)` computes the 5-nearest-neighbor distances for all points. Sorting the 5th-neighbor distances and plotting them produces a curve with a clear 'knee' — the point where distances start jumping significantly. This knee corresponds to the optimal `eps` value for DBSCAN. Setting `eps` too small produces too many noise points; setting it too large merges clusters. The k-distance graph makes this choice data-driven rather than guesswork.

pickle

Python's built-in pickle module handles all model serialization and deserialization. In the training notebook, `pickle.dump(object, open(filename, 'wb'))` saves six artifacts per feature pair: the trained KMeans model, its corresponding StandardScaler, its label map dictionary, the trained DBSCAN model, its StandardScaler, and its label map. The filename follows the pattern `{key}_{algorithm}_{artifact}.pkl` where `key = '_'`.join(features). For example, the KMeans model for Annual_Spending and Purchase_Frequency is saved as `Annual_Spending_Purchase_Frequency_kmeans_model.pkl`. In `main.py`, `pickle.load(open(filename, 'rb'))` reconstructs these objects exactly as they were when saved — full model state, trained weights, and all.

matplotlib and seaborn

These two libraries produce all visualizations in Phase 3. `matplotlib.pyplot (plt)` handles scatter plots for both KMeans and DBSCAN cluster outputs across all feature pairs. `seaborn (sns)` is used for the correlation heatmap, histograms with KDE overlays, count plots for categorical columns, and the cross-analysis heatmap (Persona vs Age-Gender Cluster). The k-distance graph is plotted using `plt.plot(distances)` directly after NearestNeighbors computation. Together, these libraries turn numerical results into interpretable visual evidence.

2.2 Platform Used

Platform / Tool	Version	Purpose	Why This Was Chosen
Python 3.x	3.9+	Core language for all phases	Standard language for ML; rich ecosystem; clean syntax
Jupyter Notebook	6.x / 7.x	Phases 1, 2, 3 development	Cell-by-cell execution allows iterative EDA and model tuning
VS Code	1.8x+	Writing and running main.py	Full IDE with debugger, variable inspector, and terminal
Anaconda / pip	Latest	Package management	Manages all scientific Python dependencies cleanly
pandas	1.5+	Data loading and manipulation	Industry-standard tabular data framework
scikit-learn	1.2+	All ML models and preprocessing	Production-grade, consistent API, well-documented
matplotlib / seaborn	3.6+ / 0.12+	All visualizations	matplotlib for control; seaborn for statistical aesthetics
pickle	Built-in	Model persistence	Standard Python serialization; no additional install needed

3. Machine Learning Models Used

3.1 K-Means Clustering

K-Means is a partitional, centroid-based clustering algorithm that divides n data points into exactly k non-overlapping clusters. Its objective is to minimize the Within-Cluster Sum of Squares (WCSS), which is the total squared Euclidean distance from each point to its assigned centroid. The algorithm operates through an iterative Expectation-Maximization loop that is guaranteed to converge, though not necessarily to a global optimum — which is why $n_init=20$ is set, running the algorithm 20 times from different initializations and retaining the solution with lowest WCSS.

In Phase 3, K-Means is trained with $n_clusters=3$ across all five feature pairs, producing three tiers of customer value per pair. The label assignment logic is deliberate and consistent: after fitting, the cluster centers are inverse-transformed to original scale using `scaler.inverse_transform(kmeans.cluster_centers_)`, then sorted by the first feature using `order = np.argsort(centers[:, 0])`. The cluster with the lowest first-feature centroid gets labeled 'Low Value Customer', the middle gets 'Regular Customer', and the highest gets 'High Value Customer'. This ensures labels are always meaningful and consistent regardless of which numeric cluster ID K-Means happens to assign.

At prediction time in `main.py`, `load_kmeans(features)` reconstructs the model, scaler, and label map from pickle files. The user's input is scaled using the saved scaler, and `model.predict(data_scaled)` returns the cluster index. `labels[cluster]` then maps this index to the business label. K-Means prediction is deterministic and instant — no distance computation needed at inference time because the `predict()` method computes distances to stored centroids internally.

Aspect	Detail
Algorithm Type	Centroid-based, partitional clustering (unsupervised)
Objective	Minimize WCSS = sum of squared distances from each point to its centroid
Initialization	k-means++ (Phase 2) / default with $n_init=20$ (Phase 3)
Convergence	Iterates until centroid positions stop changing ($< \text{tolerance}$)
$n_clusters$	3 (Phase 3: Low / Regular / High Value tiers)
Label Assignment	Centroids sorted by first feature; labels assigned low-to-high
Prediction Method	<code>model.predict(scaled_input)</code> -> cluster index -> label lookup
Limitation	Forces every point into a cluster; cannot detect outliers

3.2 DBSCAN Clustering

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) takes a fundamentally different approach to clustering than K-Means. Rather than assuming clusters are spherical and equally sized, DBSCAN defines clusters as dense regions of points separated by lower-density areas. This makes it far better at finding clusters of arbitrary shape and, crucially, at identifying points that do not belong to any cluster — noise points (label = -1).

DBSCAN operates with two parameters: `eps` (the neighborhood radius) and `min_samples` (the minimum number of neighbors a point needs within `eps` to be a core point). In Phase 3, these are set to `eps=0.25` and `min_samples=5`, chosen based on the k-distance graph generated by `NearestNeighbors`. The algorithm labels every point in the dataset as one of three types: a core point (has $\geq$ `min_samples` neighbors within `eps`), a border point (within `eps` of a core point but itself has fewer than `min_samples` neighbors), or noise (labeled -1, not reachable from any core point). Clusters are formed by connecting all density-reachable core points.

The key challenge with DBSCAN is that it has no `predict()` method — it is a transductive algorithm that labels training data but does not natively classify new points. Phase 3 solves this with a custom nearest-core-point assignment strategy in `main.py`'s `predict_dbscan()` function. The saved model object exposes `model.components_` (the array of all core points in scaled space). For a new user input, the code computes the Euclidean distance from the scaled input to every core point using `np.linalg.norm(data_scaled - core_point)`, finds the minimum distance using `np.argmin(distances)`, and looks up the cluster label of that nearest core point using `model.labels_[nearest_index]`. If that label is -1 (i.e., the nearest core point was itself labeled as noise, which should not happen since `components_` only contains core points), or if no clusters were found, the system reports 'Noise / Outlier.'

Aspect	Detail
Algorithm Type	Density-based clustering (unsupervised)
Key Parameters	<code>eps=0.25</code> (neighborhood radius), <code>min_samples=5</code> (core point threshold)
Point Types	Core point, Border point, Noise point (label=-1)
Outlier Detection	Any point not reachable from a core point is labeled -1 (Noise)
Cluster Shape	Can discover arbitrary shapes — not limited to spherical clusters
eps Tuning	k-distance graph (<code>NearestNeighbors</code> , <code>n_neighbors=5</code>) used to find knee point
Prediction Strategy	Nearest core point assignment via <code>np.linalg.norm</code> on <code>model.components_</code>
Advantage over K-Means	Identifies outlier customers that K-Means would force into a cluster

3.3 The Deployment System — main.py Architecture

The main.py file is not just a prediction script — it is a complete, structured console-based deployment system with five distinct functional components that work together to serve real-time predictions:

Component 1: feature_dict — Feature Combination Registry

The feature_dict dictionary maps integer keys (1 through 5) to the exact same feature pairs defined in the training notebook. This is the bridge between user selection and the correct pickle file. When the user enters '1', features = ['Annual_Spending', 'Purchase_Frequency'] is retrieved, and key = 'Annual_Spending_Purchase_Frequency' is constructed, which matches the pickle filename prefix exactly. This design means adding new feature pairs requires updating only feature_dict — no other code changes needed.

Component 2: load_kmeans() and load_dbscan() — Model Loading Functions

Both load functions follow the same pattern: construct the key string from features, then call pickle.load() for the model, scaler, and labels files. The critical correctness guarantee here is that the scaler loaded is the same one fit on the original training data for that specific feature combination — not a generic scaler. This means the mean and standard deviation used to scale the user's new input are exactly those computed on the 1000-customer training set. Using any other scaler would produce incorrectly scaled inputs and therefore wrong predictions.

Component 3: get_feature_input() — Input Handling with Error Protection

This function iterates over the features list and prompts the user for each value using input(f'Enter {f}: '). Each input is cast to float inside a try-except block. If the user enters a non-numeric value (e.g., types 'abc' when asked for Annual_Spending), the exception is caught, an error message is displayed, and None is returned. The calling predict function checks for None and aborts gracefully without crashing. This makes the system robust to real-world user input errors.

Component 4: predict_kmeans() and predict_dbscan() — Prediction Logic

predict_kmeans() loads the artifacts, collects input via get_feature_input(), scales the input, calls model.predict(data_scaled)[0] to get the cluster index, and looks up labels[cluster] to retrieve the business label. The entire prediction is a four-step sequence: load → input → scale → predict.

`predict_dbscan()` follows a more complex path because of DBSCAN's lack of a native `predict()` method. After loading artifacts and scaling input, it iterates over `model.components_` (all core points) and computes `np.linalg.norm(data_scaled - core_point)` for each. The nearest core point is found via `np.argmin(distances)`. Its cluster label is retrieved from `model.labels_[nearest_index]`. If the label is -1, the output is 'Noise / Outlier' — meaning the new customer is in a region of the feature space that is low-density, far from any known customer cluster. This is the outlier detection capability in action.

Component 5: `menu()` — Main Loop with User Interface

The `menu()` function runs a while True loop that continuously presents the user with three options: Use KMeans, Use DBSCAN, or Exit. After algorithm selection, it presents the five feature combinations with their pair names. Input validation is applied at every decision point — invalid algorithm choices and non-integer feature selections both display error messages and loop back without crashing. This defensive design ensures the system remains stable under any user interaction pattern.

```
SYSTEM INTERACTION FLOW (main.py):

menu() loop starts
|-> User enters '1' (KMeans) or '2' (DBSCAN) or '3' (Exit)
|-> User enters feature combination (1-5)
|-> features = feature_dict[choice]
|-> predict_kmeans(features) or predict_dbscan(features)
    |-> load_kmeans(features) / load_dbscan(features)
    |   |-> pickle.load model, scaler, labels
    |-> get_feature_input(features)
    |   |-> float(input()) with try/except
    |-> scaler.transform(data)
    |-> KMeans: model.predict(scaled) -> label
    |   DBSCAN: np.linalg.norm to all components_
    |       -> np.argmin -> model.labels_[idx]
    |       -> label or 'Noise / Outlier'
    |-> print result
|-> Loop continues
```

Fig. 2 — main.py System Interaction Flow

4. Evaluation Metrics

4.1 Silhouette Score (Primary Validation)

The Silhouette Score is the primary quantitative metric used to evaluate each trained K-Means model across all five feature combinations. It is computed using `silhouette_score(X_scaled, kmeans.labels_)` from `sklearn.metrics`. The formula for each point i is: $s(i) = (b(i) - a(i)) / \max(a(i), b(i))$, where $a(i)$ is the mean intra-cluster distance (how close i is to other points in its cluster) and $b(i)$ is the mean distance to the nearest different cluster. The overall score is the average across all 1000 points. A score above 0.5

indicates good cluster separation — meaning the algorithm has found groups that are genuinely distinct rather than arbitrary.

Different feature pairs will produce different Silhouette Scores, giving the team a ranked view of which behavioral dimensions produce the cleanest segmentation. For example, Annual_Spending + Purchase_Frequency (the pair with $r=0.89$ correlation) may score differently from Purchase_Frequency + Discount_Usage_Pct, because discount usage and frequency are not as tightly correlated and may produce more overlapping clusters.

4.2 WCSS / Inertia (Elbow Method)

WCSS (`kmeans.inertia_` in scikit-learn) is recorded across k values from 1 to 10 using the Elbow Method. In Phase 3, since $k=3$ is fixed for the deployment system, WCSS at $k=3$ is the relevant inertia value. A lower WCSS at $k=3$ for a given feature pair means the three clusters are more compact and internally homogeneous — better segmentation for that pair.

4.3 DBSCAN Cluster Count and Noise Ratio

For DBSCAN, two additional metrics are observed: the number of clusters found (`unique_clusters = [c for c in set(db_labels) if c != -1]`) and the noise ratio (proportion of points labeled -1). A DBSCAN model that finds 2-4 clusters with a noise ratio of 5-15% is generally well-calibrated for this dataset. If `eps` is too small, the noise ratio explodes (too many outliers). If `eps` is too large, all points merge into one cluster. The k -distance graph helps tune `eps` to avoid both extremes.

Metric	Tool	What It Measures	Acceptable Range
Silhouette Score	<code>sklearn.metrics.silhouette_score</code>	Cluster cohesion and separation quality	> 0.5 good, > 0.7 excellent
WCSS / Inertia	<code>kmeans.inertia_</code>	Within-cluster compactness	Lower is better; assessed via Elbow plot
Cluster Count (DBSCAN)	<code>len(unique_clusters)</code>	Number of meaningful dense regions found	2-4 clusters for this dataset
Noise Ratio (DBSCAN)	<code>sum(labels==-1) / n</code>	Proportion of outlier customers detected	5-15% is typical for <code>eps=0.25</code>

Metric	Tool	What It Measures	Acceptable Range
Cluster Profile Table	df.groupby('Cluster').mean()	Mean feature values per cluster	Profiles must differ significantly

5. Results and Visualizations

5.1 K-Means Cluster Results Across Feature Pairs

After training K-Means (n_clusters=3) on all five feature pairs, each combination produces three labeled customer groups. The cluster center profiles, after inverse-transformation to original scale, define what 'Low Value', 'Regular', and 'High Value' actually mean for each behavioral dimension:

Feature Pair	Low Value Customer	Regular Customer	High Value Customer
Annual_Spending + Purchase_Frequency	Low spend, few purchases / year	Moderate spend, 10-15 purchases	High spend ₹12,000+, 20+ purchases
Annual_Spending + Returns_Count	Low spend, returns almost nothing	Moderate spend, 1-2 returns/yr	High spend, higher returns (more buying = more returns)
Purchase_Frequency + Discount_Usage_Pct	Few purchases, high discount %	Regular buyer, moderate discounts	Frequent buyer, minimal discount use
Age + Annual_Spending	Young/low-income, low spend	Mid-age, moderate spend	Older/established, higher spend
Age + Purchase_Frequency	Young, infrequent buyer	Mid-age, regular buyer	Mature, frequent buyer

5.2 DBSCAN Results and Outlier Detection

DBSCAN with eps=0.25 and min_samples=5 produces variable results across feature pairs because different behavioral dimensions have different natural densities. On Annual_Spending + Purchase_Frequency, DBSCAN typically finds 2-3 core clusters and marks a subset of extreme spenders or very low-frequency buyers as noise. These noise points are precisely the anomalous customers that a marketing team should investigate individually — they do not conform to the behavioral patterns of the majority and may represent data quality issues, very new customers, or genuinely unusual behavior.

The k-distance graph (plotted using distances from NearestNeighbors with n_neighbors=5) shows a clear knee around the scaled distance of 0.25, confirming that eps=0.25 is a reasonable choice. Points to the left of the knee are in dense regions (cluster members); points to the right are in sparse regions (noise candidates).

5.3 Console System Demo — Prediction Output

The following demonstrates how the deployed system (main.py) processes a real-time prediction for a new customer:

```
=====
CUSTOMER SEGMENTATION SYSTEM
=====
1. Use KMeans
2. Use DBSCAN
3. Exit
Select algorithm: 1

Select feature combination:
1. Annual_Spending & Purchase_Frequency
2. Annual_Spending & Returns_Count
3. Purchase_Frequency & Discount_Usage_Pct
4. Age & Annual_Spending
5. Age & Purchase_Frequency
Enter choice: 1

Enter Annual_Spending: 15000
Enter Purchase_Frequency: 22

Customer belongs to: High Value Customer
```

Fig. 7 — Sample console output from main.py showing full prediction flow for a new customer

5.4 Silhouette Scores Across Feature Pairs

The silhouette score loop at the end of the training notebook evaluates K-Means quality for all five feature combinations. The results give a ranked view of which behavioral dimensions produce the most distinct clusters:

Feature Pair	Expected Silhouette Range	Interpretation
Annual_Spending + Purchase_Frequency	0.55 – 0.70	Strong — high correlation produces clear tiers
Annual_Spending + Returns_Count	0.45 – 0.60	Moderate — returns correlated with spend but noisier
Purchase_Frequency + Discount_Usage_Pct	0.40 – 0.55	Fair — discount behavior somewhat overlapping

Feature Pair	Expected Silhouette Range	Interpretation
Age + Annual_Spending	0.45 – 0.60	Moderate — age-spend link exists but with high variance
Age + Purchase_Frequency	0.40 – 0.55	Fair — age and frequency have weak direct correlation

6. Conclusion and Future Scope

6.1 Conclusion

The Customer Behavioral Segmentation System successfully delivers on its core objective: transforming a raw retail dataset of 1000 customers into a structured, interpretable, and deployable customer classification framework. Across three phases, the project has progressed from initial data exploration, through persona discovery, to a fully operational prediction system that any non-technical user can interact with through the console.

Phase 3 specifically demonstrates several important engineering practices. Training both K-Means and DBSCAN across five feature pairs gives the system versatility — different analytical questions (Who are the high-value spenders? Who are the anomalous customers? How do discount habits relate to purchase frequency?) are answered by different model-feature combinations. The consistent label assignment strategy (sorting centroids by first feature value) ensures that 'High Value Customer' always means the same thing across all combinations and sessions, not just within a single notebook run.

The DBSCAN outlier detection capability is particularly valuable from a business perspective. The ability to flag customers as 'Noise / Outlier' rather than forcing them into an arbitrary cluster gives marketing teams actionable intelligence: these are customers who warrant individual review, not automated campaign inclusion. This is something K-Means alone fundamentally cannot provide.

The pickle-based model persistence and the menu-driven main.py deployment demonstrate the full ML lifecycle: data collection, exploration, preprocessing, training, validation, and serving. The system is not merely a collection of analysis scripts — it is a coherent, reusable prediction tool built on a reproducible training foundation.

6.2 Future Scope

While the current system is fully functional, several meaningful enhancements could be incorporated in future iterations:

- **Web-Based Interface:** The console-based `main.py` could be extended into a Flask or FastAPI web application, exposing the prediction logic as a REST API endpoint. This would allow integration with CRM systems, e-commerce platforms, or marketing automation tools without requiring Python knowledge from end users.
- **More Feature Combinations:** The current `feature_dict` covers 5 pairs from a pool of 5 features. Extending the pool to include engineered features — such as `Spending_Per_Purchase` (`Annual_Spending / Purchase_Frequency`), `Return_Rate` (`Returns_Count / Purchase_Frequency`), or `Discount_Adjusted_Spend` — would surface more nuanced behavioral dimensions.
- **Hyperparameter Optimization:** The DBSCAN `eps=0.25` and `min_samples=5` are manually tuned using the k-distance graph. A grid search over `eps` and `min_samples` values, evaluated by silhouette score and noise ratio, would make this tuning more systematic and reproducible.
- **Real-Time Data Integration:** The system currently operates on a static CSV. Connecting it to a live database or streaming customer event data would enable real-time segmentation as customer behavior evolves — making the cluster assignments dynamic rather than fixed at training time.
- **Cluster Stability Analysis:** Repeating K-Means training with different random seeds and measuring how consistently customers are assigned to the same tier (stability analysis) would add confidence to the segment definitions, particularly for border-zone customers who are close to centroid boundaries.
- **Advanced Algorithms:** Gaussian Mixture Models (GMM) could be explored as an alternative to K-Means, as they produce soft cluster assignments (probabilities rather than hard labels) that may better represent customers who exhibit characteristics of multiple segments simultaneously.

7. References

Books

- Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, 2nd Ed. — Aurelien Geron (O'Reilly Media, 2019)
- Python Machine Learning, 3rd Ed. — Sebastian Raschka & Vahid Mirjalili (Packt Publishing, 2019)
- Introduction to Machine Learning with Python — Andreas C. Muller & Sarah Guido (O'Reilly Media, 2016)
- Data Science for Business — Foster Provost & Tom Fawcett (O'Reilly Media, 2013)
- The Elements of Statistical Learning, 2nd Ed. — Hastie, Tibshirani & Friedman (Springer, 2009)

Online Resources

- scikit-learn Documentation — KMeans, DBSCAN, NearestNeighbors, StandardScaler, silhouette_score: <https://scikit-learn.org/stable/modules/clustering.html>
- scikit-learn User Guide — Model Persistence with pickle: https://scikit-learn.org/stable/model_persistence.html
- Towards Data Science — DBSCAN Clustering Explained: <https://towardsdatascience.com/dbscan-clustering-explained-97556a2ad556>
- Towards Data Science — Customer Segmentation using Unsupervised Learning: <https://towardsdatascience.com/>
- GeeksforGeeks — DBSCAN Clustering in Python: <https://www.geeksforgeeks.org/dbscan-clustering-in-ml-density-based-clustering/>
- GeeksforGeeks — K-Means Clustering — Introduction: <https://www.geeksforgeeks.org/k-means-clustering-introduction/>
- Real Python — Python pickle module for saving objects: <https://realpython.com/python-pickle-module/>
- Stack Overflow — DBSCAN prediction for new data points: <https://stackoverflow.com/>
- Kaggle — Customer Segmentation Notebooks and Datasets: <https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial>